



**TechBiz Academy**

**Week 5\_Task 14**

**Cross-Site Scripting (XSS) — DVWA**

**Submitted By**

**Ali Hamza**

**Registration No**

**TBS-2026-017**

**Course**

**Ethical Hacking Internship**

**Submission Date**

**31 July 2026**

**Organization**

**TechBizz Security**

# Table of Contents

| Contents                                        | Page No. |
|-------------------------------------------------|----------|
| Cover Page                                      | i        |
| Table of Contents                               | ii       |
| 1. Objective                                    | 1        |
| 2. Lab Environment                              | 2        |
| 3. Task 1: Reflected Cross-Site Scripting (XSS) | 3        |
| 3.1 Objective                                   | 3        |
| 3.2 Procedure                                   | 3        |
| 3.3 Observation                                 | 4        |
| 3.4 Evidence (SSC-1 & SSC-2)                    | 4        |
| 3.5 Result                                      | 5        |
| 4. Task 2: Stored Cross-Site Scripting (XSS)    | 6        |
| 4.1 Objective                                   | 6        |
| 4.2 Procedure                                   | 6        |
| 4.3 Observation                                 | 7        |
| 4.4 Evidence (SSC-3)                            | 7        |
| 4.5 Result                                      | 8        |
| 5. Difference Between Reflected and Stored XSS  | 9        |
| 6. Remediation Techniques                       | 10       |
| 6.1 Input Validation                            | 10       |
| 6.2 Output Encoding                             | 11       |

| Contents                                | Page No. |
|-----------------------------------------|----------|
| 6.3 Additional Security Recommendations | 11       |
| 7. Learning Outcomes                    | 12       |
| 8. Conclusion                           | 13       |
| 9. References                           | 14       |

## Objective

The objective of this practical exercise was to understand the behavior of Cross-Site Scripting (XSS) vulnerabilities using the Damn Vulnerable Web Application (DVWA). The assessment focused on both Reflected XSS and Stored XSS vulnerabilities in an authorized laboratory environment. Additionally, the exercise aimed to understand the key differences between these vulnerability types and study common remediation techniques such as input validation and output encoding.

## Lab Environment

| Component           | Details                                |
|---------------------|----------------------------------------|
| Platform            | Damn Vulnerable Web Application (DVWA) |
| Operating System    | Kali Linux                             |
| Web Server          | Apache                                 |
| Database            | MySQL / MariaDB                        |
| Security Level      | Low                                    |
| Testing Environment | Authorized Local Lab                   |

## Task 1: Reflected Cross-Site Scripting (XSS)

### Objective

To verify the existence of a Reflected Cross-Site Scripting vulnerability within the DVWA application.

## Procedure

1. Logged into the DVWA application.
2. Set the DVWA security level to **Low**.
3. Opened the **XSS (Reflected)** module.
4. Entered the JavaScript test input into the available form field.
5. Submitted the request and observed the application's behavior.

---

## Observation

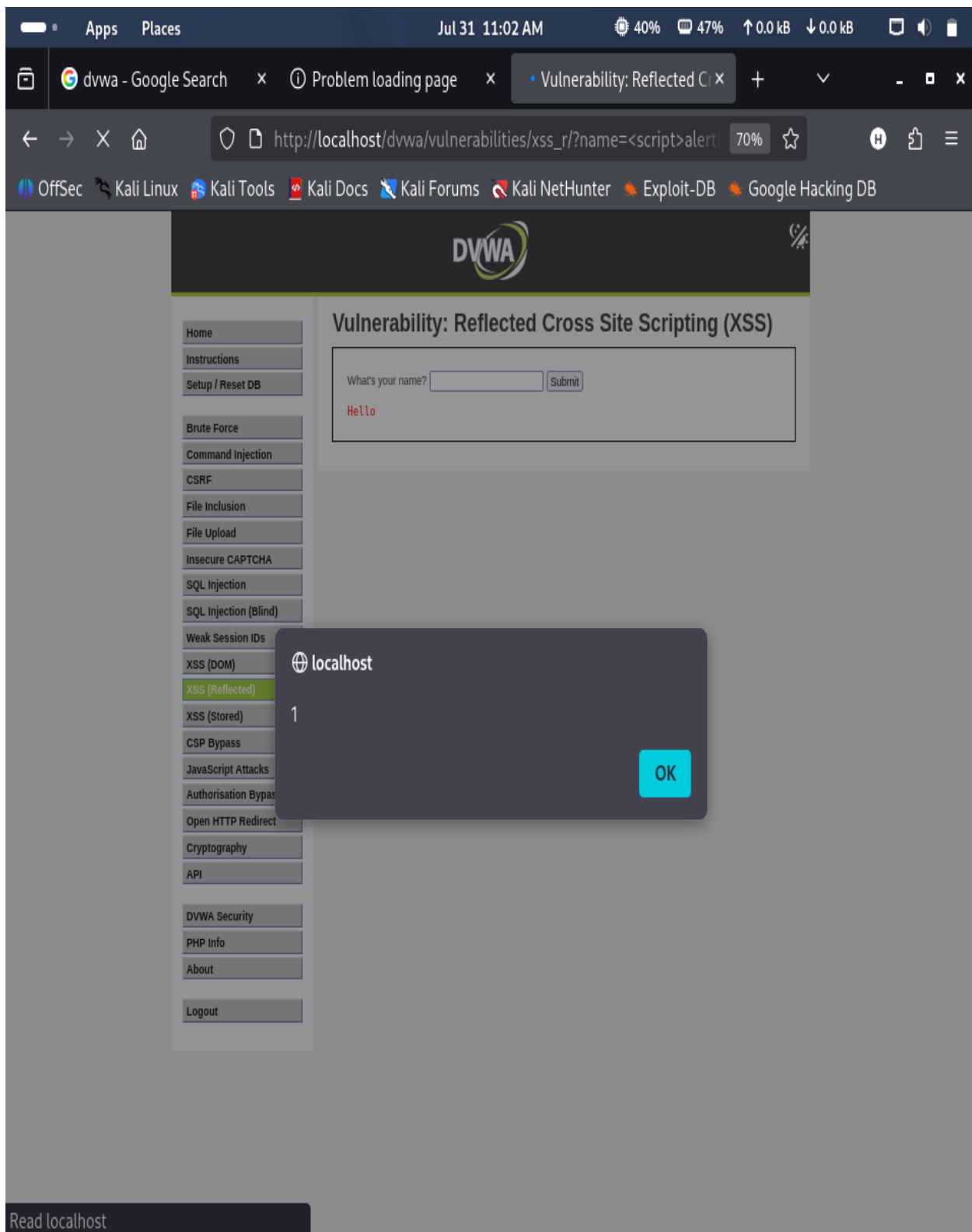
The application reflected the submitted input back to the browser without applying appropriate output encoding. As a result, the browser executed the JavaScript code, confirming the presence of a Reflected XSS vulnerability.

---

## Evidence

### SSC-1: Reflected XSS Alert

*(Screenshot showing the JavaScript alert displayed after the test input.)*



## SSC-2: Application After Alert

(Screenshot showing the webpage after closing the alert dialog.)

The screenshot shows a web browser window with the URL `http://localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert(1)`. The browser's address bar shows the URL and a 70% zoom level. The browser's taskbar at the bottom includes icons for OffSec, Kali Linux, Kali Tools, Kali Docs, Kali Forums, Kali NetHunter, Exploit-DB, and Google Hacking DB.

The DVWA application interface is displayed. The left sidebar contains a list of navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), **XSS (Reflected)**, XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, Open HTTP Redirect, Cryptography, API, DVWA Security, PHP Info, About, and Logout. The **XSS (Reflected)** link is highlighted in green.

The main content area is titled "Vulnerability: Reflected Cross Site Scripting (XSS)". It features a form with the label "What's your name?" and a "Submit" button. Below the form, the text "Hello" is displayed in red. Under the "More Information" section, there are five links: <https://owasp.org/www-community/attacks/xss/>, <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>, [https://en.wikipedia.org/wiki/Cross-site\\_scripting](https://en.wikipedia.org/wiki/Cross-site_scripting), <https://www.cgisecurity.com/xss-faq.html>, and <https://www.scriptalert1.com/>.

At the bottom of the page, the status information is displayed: Username: admin, Security Level: Security Level: low, Locale: en, and SQLi DB: mysql. There are "View Source" and "View Help" buttons. The footer text reads "Damn Vulnerable Web Application (DVWA)".

## Result

The Reflected XSS vulnerability was successfully demonstrated in the authorized DVWA laboratory environment.

---

## Task 2: Stored Cross-Site Scripting (XSS)

### Objective

To understand how malicious user input can be permanently stored and later executed when viewed.

---

### Procedure

1. Opened the **Stored XSS** module.
  2. Entered sample data into the Guestbook.
  3. Submitted the information.
  4. Observed how the submitted content appeared on the webpage after storage.
- 

### Observation

The submitted content was successfully stored by the application and displayed whenever the Guestbook page was viewed. This demonstrates how Stored XSS differs from Reflected XSS by persisting within the application until removed.

---

### Evidence

## SSC-3: Guestbook Submission

(Screenshot showing the Guestbook submission page.)

The screenshot shows a web browser window with the URL `http://localhost/dvwa/vulnerabilities/xss_s/`. The page title is "Vulnerability: Stored Cross Site Scripting (XSS)". The left sidebar contains a list of vulnerability categories, with "XSS (Stored)" highlighted. The main content area shows a form with the following fields:

- Name \*:
- Message \*:
- Buttons:

Below the form, there are two preview boxes showing the submitted message:

```
Name: test
Message: This is a test comment.

Name: Ali Hamza
Message: i am a professional of cyber security
```

Under the "More Information" section, there are five links:

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- [https://en.wikipedia.org/wiki/Cross-site\\_scripting](https://en.wikipedia.org/wiki/Cross-site_scripting)
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

At the bottom of the page, there is a footer with the text "Damn Vulnerable Web Application (DVWA)".

---

## Result

The Stored XSS behavior was successfully observed in the DVWA laboratory environment.

---

# Comparison Between Reflected and Stored XSS

### Reflected XSS

User input is reflected immediately within the server response.

The attack executes only after a specially crafted request is processed.

The malicious data is not permanently saved.

Generally affects users interacting with a crafted request.

### Stored XSS

User input is permanently stored within the application database or storage.

The attack executes whenever the stored content is viewed.

The malicious data persists until it is removed.

Can potentially affect every user who accesses the stored content.

---

## Remediation Techniques

### 1. Input Validation

Input validation ensures that applications only accept data matching expected formats and lengths. Rejecting unexpected or malformed input reduces the likelihood of malicious content entering the application.

---

### 2. Output Encoding

Output encoding converts special characters into safe representations before displaying user-supplied content in the browser. This prevents browsers from interpreting user input as executable HTML or JavaScript.

---

## Additional Security Recommendations

- Validate all user input on the server side.
  - Encode output before rendering dynamic content.
  - Implement a strong Content Security Policy (CSP).
  - Keep web frameworks and third-party libraries up to date.
  - Perform regular security assessments and code reviews.
- 

## Resources Used

- TryHackMe — XSS Learning Room
  - OWASP Cross-Site Scripting (XSS) Documentation
  - Damn Vulnerable Web Application (DVWA)
  - Video: “XSS attack Dvwtutorial beginners”
  - Video: “stored vs reflected XSS explained”
- 

## Learning Outcomes

After completing this laboratory exercise, the following concepts were understood:

- Understanding of Reflected Cross-Site Scripting.
  - Understanding of Stored Cross-Site Scripting.
  - Identification of differences between XSS types.
  - Importance of secure coding practices.
  - Understanding of input validation.
  - Understanding of output encoding.
  - Awareness of common XSS mitigation techniques.
- 

## Conclusion

This practical exercise successfully demonstrated the behavior of Reflected and Stored Cross-Site Scripting vulnerabilities within the authorized DVWA training environment. The assessment highlighted how improper handling of user input can lead to client-side code execution and emphasized the importance of implementing secure development practices. Applying robust input validation, proper output encoding, and additional browser security mechanisms significantly reduces the risk of XSS vulnerabilities in modern web applications.